Churn Prediction Pipeline — Executive Summary

This brief synthesizes an end-to-end churn prediction pipeline built to maximize subscriber retention ROI in saturated markets. The project ingests CSV staging files into MySQL, applies robust warehouse and segmentation logic, and trains a production-ready predictive model in Python. Outcomes: improved retention targeting, automated mitigation triggers, and analyst-facing explainability for prioritized actions.

Business Problem

High acquisition costs + saturated market cause revenue erosion from preventable churn.

Project Goals

Deliver a reliable pipeline from CSV → MySQL → Python that predicts churn, explains drivers, and triggers interventions.

Key Operational Outcomes

Automated webhooks for at-risk customers, weekly retraining schedule, and dashboard-ready scores for product teams.

Tools Used

Explicit stack: MySQL (data warehouse & segmentation logic), Python (data engineering & modeling), Scikit-learn (Random Forest Classifier), ELI5 (feature explanation), and lightweight orchestration (cron or Airflow) for retraining and jobs.

High-impact Insight

Primary risk drivers surfaced by model explainability: daytime usage decline and increased customer service calls. These map directly to automated retention playbooks.

Design Principles

- Schema-first staging to prevent drift
- Embed business rules in SQL for efficient segmentation
- Prefer interpretable ensembles and post-hoc explainers

Implementation Roadmap & Operational Playbook

Below is a condensed, actionable roadmap that merges the documented steps into operational priorities for data engineers and data scientists. The goal is to move from prototype to repeatable production with monitoring and explainability baked in.

Each step maps to clear ownership: ingestion (data engineering), warehouse rules and segmentation (DBA + analytics), modeling and explainability (data science), and automation/operations (SRE/product).

Step 1 — Ingestion & Schema

Stage raw files into raw_churn_staging to protect against schema drift. Transform into a star schema with customer dimension and customer_usage_fact for fast aggregation.

Step 2 — Warehouse Logic

Implement segmentation rules in SQL to label At Risk, Loyal, and Active Standard tiers. Keep logic set-based for performance and auditability.

Step 3 — Predictive Layer

Preprocess (handle multicollinearity, drop high-cardinality geo strings), perform stratified 80/20 split, train a Random Forest ensemble, and persist model artifacts and feature pipelines.

Step 4 — Explainability & Deployment

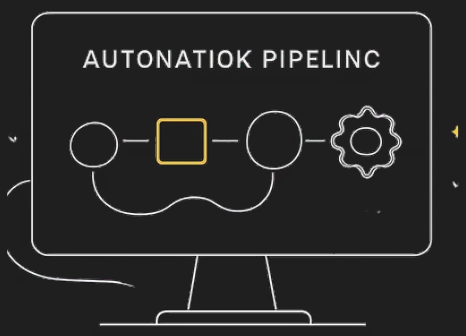
Use ELI5 to extract top drivers (e.g., daytime usage, service calls). Serve scores and explanations via a microservice that emits webhooks to retention systems and populates dashboards.

Monitoring & Ops

- Weekly model retrain job with drift checks
- Data-quality alerts on staging and fact loads
- Performance SLAs for scoring service

Next Steps & Recommendations

1. Formalize retraining cadence and holdout validation
2. Build retention playbooks mapped to top risk drivers
3. Instrument dashboards for transparency and A/B testing



Prioritize reproducibility (pipelines, schema migrations, and model versioning) — these choices determine long-term business impact.